# Properties of clauses

## Order of literals does not matter

If a clause $C$ is obtained by reordering the literals of a clause $C'$ then $C$ and $C'$ are equivalent. E.g., $(p \vee q \vee r \vee \neg r) \equiv (\neg r \vee q \vee p \vee r)$

## Multiple literals can be merged

If a clause contains more than one occurrence of the same literal, then it is equivalent to the clause obtained by deleting all but one such occurrences. E.g., $(p \vee q \vee r \vee q \vee \neg r) \equiv (p \vee q \vee r \vee \neg r)$

## Clauses as set of literals

It follows from these properties that we can represent a clause as a set of literals, by leaving disjunction implicit and by ignoring replication and order of literals

$(p \vee q \vee r \vee \neg r)$ is represented by the set $\{p, q, r, \neg r\}$

# Properties of formulas in CNF

## Order of clauses does not matter

If a CNF formula $\phi$ is obtained by reordering the clauses of a CNF formula $\phi'$ then $\phi$ and $\phi'$ are equivalent. E.g.,

$$(a \vee b) \wedge (c \vee \neg b) \wedge (\neg b) \equiv (c \vee \neg b) \wedge (\neg b) \wedge (a \vee b)$$

## Multiple clauses can be merged

If a CNF formula contains more than one occurrence of the same clause, then it is equivalent to the formula obtained by deleting all but one such occurrences. E.g.,

$$(a \vee b) \wedge (c \vee \neg b) \wedge (a \vee b) \equiv (a \vee b) \wedge (c \vee \neg b)$$

## A CNF formula can be seen as a set of clauses

It follows from the properties of clauses and CNF formulas that we can represent a CNF formula as a set of sets of literals. E.g., $(\neg b) \wedge (b \vee a \vee b) \wedge (c \vee \neg b) \wedge (\neg b)$ is represented by $\{\{a, b\}, \{c, \neg b\}, \{\neg b\}\}$

# Satisfiability of a set of clauses

- Let $\psi = \{C_1, \ldots, C_n\}$
  - $\mathcal{I} \models \psi$ if and only if $\mathcal{I} \models C_i$ for all $i = 1..n$;
  - $\mathcal{I} \models C_i$ if and only if for some $l \in C$, $\mathcal{I} \models l$
- To check if an intepretation $\mathcal{I}$ satisfies $\psi$ we do not necessarily need to know the truth values that $\mathcal{I}$ assigns to all the literals appearing in $\psi$.
- For instance, if $\mathcal{I}(p) = true$ and $\mathcal{I}(q) = false$, we can say that $\mathcal{I} \models \{\{p, q, \neg r\}, \{\neg q, s, q\}\}$, without considering the evaluations of $\mathcal{I}(r)$ and $\mathcal{I}(s)$.

**Partial interpretation**

A partial intepretation is a partial function that associates to some propositional variables of the alphabet $P$ a truth value (either true or false) and can be undefined for the others.

# Partial interpretations

- Partial interpretations can be used to construct models for a set of clauses $\psi = \{C_1, \ldots, C_n\}$ incrementally
- Under a partial interpretation $\mathcal{I}$, literals and clauses can be true, false or undefined; a clause $C$
  - is true under $\mathcal{I}$ if at least one of its literals is true;
  - is false (or "conflicting") under $\mathcal{I}$ if all its literals are false
  - is undefined (or "unresolved") under $\mathcal{I}$, otherwise.
- The algorithms that exploit partial intepretations usually start with an empty interpretation (where the truth values of all propositional letters are undefined) and try to extend it step by step to the various variables occurring in $\{C_1, \ldots, C_n\}$.

# Simplification

- If $\lambda$ is a literal of the form $p$, then $\neg\lambda$ denotes $\neg p$.
- If $\lambda$ is a literal of the form $\neg p$, then $\neg\lambda$ denotes $p$.

## Simplification of a formula by a literal

For any CNF formula $\phi$ and a literal $\lambda$, $\phi|_\lambda$ stands for the formula obtained from $\phi$ by

- removing all clauses containing the literal $\lambda$, and
- removing the literals $\neg\lambda$ in all remaining clauses

## Example

For instance,
$\{\{p, q, \neg r\}, \{\neg p, \neg r\}\}|_{\neg p} = \{\{q, \neg r\}\}$

# Unit propagation

## Unit clause

A unit clause is a clause containing a single literal.

If the CNF formula $\phi$ contains a unit clause $\{\lambda\}$, then to satisfy $\phi$ the literal $\lambda$ must be evaluated to True. As a consequence $\phi$ can be simplified using the procedure UnitPropagation, that is invoked on $\phi$ and a partial intepretation $\mathcal{I}$, and returns a CNF formula and a partial interpretation.

## Unit propagation

UnitPropagation($\phi, \mathcal{I}$)
  **while** $\phi$ contains a unit clause $\{\lambda\}$
    if $\lambda = p$, then $\mathcal{I}(p) := true$;
    if $\lambda = \neg p$, then $\mathcal{I}(p) := false$
    $\phi := \phi|_\lambda$;
  **end**;
  **return** ($\phi$,$\mathcal{I}$)

# The DPLL procedure

## Example

$$\textsc{UnitPropagation}(\{\{p\},\{\neg p,\neg q\},\{\neg q,r\}\},\emptyset)$$

$$\begin{array}{ll}
\{\{p\},\{\neg p,\neg q\},\{\neg q,r\}\} & \\
\{\{p\},\{\neg p,\neg q\},\{\neg q,r\}\}|_p = \{\{\neg q\},\{\neg q,r\}\} & \mathcal{I}(p) = true \\
\{\{\neg q\},\{\neg q,r\}\} & \\
\{\{\neg q\},\{\neg q,r\}\}|_{\neg q} = \{\} & \mathcal{I}(q) = false
\end{array}$$

the procedure returns $(\{\},\mathcal{I})$

# The DPLL procedure

## Remark

In some cases, unit propagation applied to $\phi$ is enough to decide satisfiability of $\phi$, for example when it terminates with one of the following two results:

- $(\{\}, \mathcal{I})$ (as in the example above), in which case the initial formula is satisfiable, and a satisfying interpretation can be easily extracted from $\mathcal{I}$;
- $(\phi, \mathcal{I})$, where $\{\} \in \phi$, in which case the original formula is unsatisfiable.

There are cases where UnitPropagation does terminate with any of the cases above, i.e., when there is no unit clauses to consider, the CNF is nonempty, and it does not contain empty clauses.

Example: $\{\{p, q\}, \{\neg q, r\}\}$

# The DPLL procedure

## The Davis-Putnam-Logemann-Loveland procedure

To check the CNF formula $\phi$ for satisfiability, the procedure is invoked by $\mathrm{DPLL}(\phi, \emptyset)$.

$\mathrm{DPLL}(\phi, \mathcal{I}')$
  $(\psi, \mathcal{I}) := \textsc{UnitPropagation}(\phi, \mathcal{I}')$;
  **if** $\psi$ contains $\{\}$
  **then** return $(\{\{\}\}, \emptyset)$
  **elseif** $\psi = \{\}$ **then** return $(\{\}, \mathcal{I})$
  **else** select a literal $\lambda \in C \in \psi$;
    **if** $\mathrm{DPLL}(\psi \cup \{\{\lambda\}\}, \mathcal{I}) = (\{\}, \mathcal{I}'')$
    **then** return $(\{\}, \mathcal{I}'')$
    **else** return $\mathrm{DPLL}(\psi \cup \{\{\neg\lambda\}\}, \mathcal{I})$

- UnitPropagation realizes the "unit propagation rule"
- The last "if then else" realizes the "splitting rule"

# Properties of the DPLL procedure

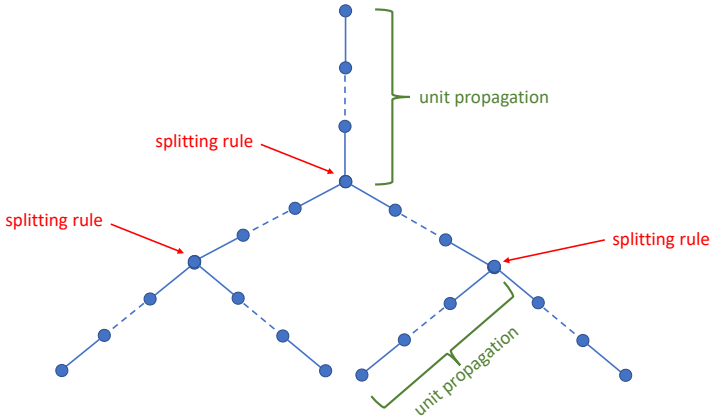
We first discuss the properties of UnitPropagation. By observing that every execution of the instructions inside the while loop eliminates one clause from the formula and possibly a set of literals from the other clauses of the input formula, we can easily conclude the following.

**Theorem**

For any $\phi, \mathcal{I}$, UnitPropagation$(\phi, \mathcal{I})$ terminates, and runs in polynomial time with respect to the size of $\phi$.

## Derivation tree

The computation done by $\mathrm{DPLL}(\phi, \emptyset)$ can be described by a binary tree, in which every path from the root to a splitting node, or from a splitting node to another splitting node, or from a splitting node to a leaf is the sequence of operations of unit propagation.



# Properties of the DPLL procedure

Since the length of every path from the root to a leaf is bound to $n$, where $n$ is the number of variables in $\phi$, we have that the size of the binary tree is at most $2^n$. From this observation the following theorem on the worst-case time complexity follows.

**Theorem**

For any $\phi$, $\mathrm{DPLL}(\phi, \emptyset)$ terminates, and its time complexity is $O(2^m)$, where $m$ is the size of $\phi$.

# Properties of the DPLL procedure

The following theorem sanctions the correctness of $\mathrm{DPLL}$.

**Theorem**

For any $\phi$, $\mathrm{DPLL}(\phi, \emptyset)$ returns

- $(\{\}, \mathcal{I})$ if $\phi$ is satisfiable, and
- $(\{\{\}\}, \emptyset)$ if $\phi$ is unsatisfiable.

Note that when $\mathrm{DPLL}(\phi, \emptyset)$ returns $(\{\}, \mathcal{I})$, $\mathcal{I}$ is a model of $\phi$.

# Horn clauses

## Definition

A clause is a Horn clause if it has at most one positive literal. A Horn formula is a formula in CNF all of whose clauses are Horn clauses.

## Theorem

A Horn formula $\phi$ is satisfiable if and only if $\textsc{UnitPropagation}(\phi, \emptyset)$ returns $(\psi, \mathcal{I})$, with $\psi$ different from $\{\{\}\}$.

## Theorem

Satisfiability of Horn formulas can be solved in polynomial time.

We leave as an exercise the proof of the above theorems.

# Exercises on Knowledge Representation and Reasoning in Propositional Logic

Giuseppe De Giacomo

Sapienza Università di Roma

Courtesy of Chiara Ghidini, Maurizio Lenzerini, Daniele Nardi, Stuart Russell